

St. Francis Institute of Technology, Mumbai-400 103
Department Of Information Technology

A.Y. 2025-2026

Class: BE-ITA/B, Semester: VIII

Subject: BlockChain Lab

Experiment 4

1. **Aim:** To implement smart contract using Remix IDE and deploy it using Ganache and MetaMask.
2. **Objective:** To ...
 - explore working of Ganache ethernet network.
 - test a smart contract by using Remix IDE
3. **Lab outcome:** After performing the experiment, the students will be able to **implement** smart contracts in Ethereum using different development frameworks (PO3, PSO2, BL3)
4. **Prerequisite:**
 - Fundamental knowledge of blockchain
 - Knowledge of the Ethereum platform and Remix IDE
 - Familiarity with the Solidity programming language
5. **Requirements:** The following are the requirements –
Remix IDE, Ganache Provider, MetaMask wallet etc.

6. Pre-Experiment Theory:

What is Ganache?

Ganache is a private Ethereum blockchain development environment that allows you to emulate the Ethereum blockchain so that you can interact with smart contracts in your own private blockchain. You can use Ganache across the entire development cycle; enabling you to develop, deploy, and test your dApps (decentralized Applications) in a safe and deterministic environment.

7. Laboratory Exercise

A. Steps to be implemented.

To Follow the procedure given below to build smart contract in Remix IDE

- Download and install Ganache from <https://truffleframework.com/ganache>.
- Download and install the official MetaMask extension/ plugin.
- Open Remix IDE in Google Chrome.
- Write smart contract by creating new file under contract folder, with .sol extension.
- Write your contract code using solidity language.
- Click on compiler icon, choose compiler version or keep default and click on compile button.
- For deployment...

Part A) First start Ganache provider and then in Remix IDE, choose environment Ganache Provider and then deploy the said contract.

Part B) Set-up Ganache with MetaMask and then in Remix IDE choose environment Injected Provider- MetaMask and then deploy the said contract.

- Under deployed contract, get the output of your contract.

B. Program Code

Write a smart contract StudentMarks.sol to create student database with attributes, id,name, dept, sub1marks, sub2Marks etc. and deploy it using Ganache and MetaMask.

8. Post Experimental Exercise-**A. Questions:**

1. List down the details of Ganache IDE
2. Create any smart contract that you have not done so far and deploy it.

B. Results/Observations/Program output:

Present the program input/output results if any and comment on the same.

C. Conclusion:

- Write what was performed in the experiment.
- Write which tools you used to perform the experiment
- Write what you inferred from the output obtained.

D. References:

- <https://www.geeksforgeeks.org/how-to-use-metamask-to-deploy-a-smart-contract-in-solidity-blockchain/>
- <https://www.geeksforgeeks.org/how-to-set-up-ganche-with-metamask/>
- Mastering Ethereum, Building Smart Contract and Dapps, Andreas M. Antonopoulos Dr. Gavin Wood, O'reilly



DEPLOY & RUN TRANSACTIONS

ENVIRONMENT

Dev - Ganache Provider

Geth Testnet (1337) network

ACCOUNT

0xc9b...465d3 (99.9998608269)

GAS LIMIT

Estimated Gas

Custom 3000000

VALUE

0 Wei

CONTRACT

Storage - contracts/1_Storage.sol

evm version: london

Verify Contract on Explorers

Deploy

At Address Load contract from Address

Configuring Ganache provider as environment

[block:1 txIndex:0] from: 0xc9b...465d3 to: Storage.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xed7...eae33

status 1 Transaction mined and execution succeed

transaction hash 0x2b1341abd0b14804cca8a98478713b8fc00e2992b3af5c50a009812d45cd72ed

block hash 0xed700e9c7e851e83de5eaca66f8e85b27e11545e3c747244d1f43908e0eae33

block number 1

contract address 0x5ca793e92657055c1e35cff9ba226779ef48b164

from 0xc9b16731f10547d024e7341644c20c97cd0465d3

to Storage.(constructor)

transaction cost 125673 gas

decoded input {}

decoded output -

logs {}

raw logs {}

Deployed and ran the transaction

[block:1 txIndex:0] from: 0xc9b...465d3 to: Storage.(constructor) value: 0 wei data: 0x608...f0033 logs: 0 hash: 0xed7...eae33

status 1 Transaction mined and execution succeed

transaction hash 0x2b1341abd0b14804cca8a98478713b8fc00e2992b3af5c50a009812d45cd72ed

block hash 0xed700e9c7e851e83de5eaca66f8e85b27e11545e3c747244d1f43908e0eae33

block number 1

contract address 0x5ca793e92657055c1e35cff9ba226779ef48b164

from 0xc9b16731f10547d024e7341644c20c97cd0465d3

to Storage.(constructor)

transaction cost 125673 gas

decoded input {}

decoded output -

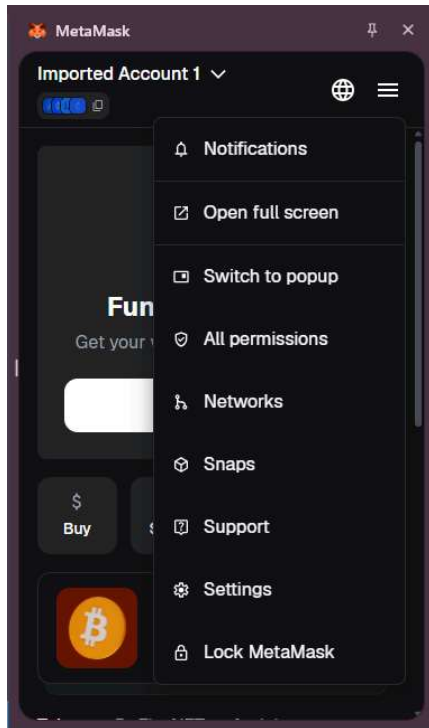
logs {}

raw logs {}

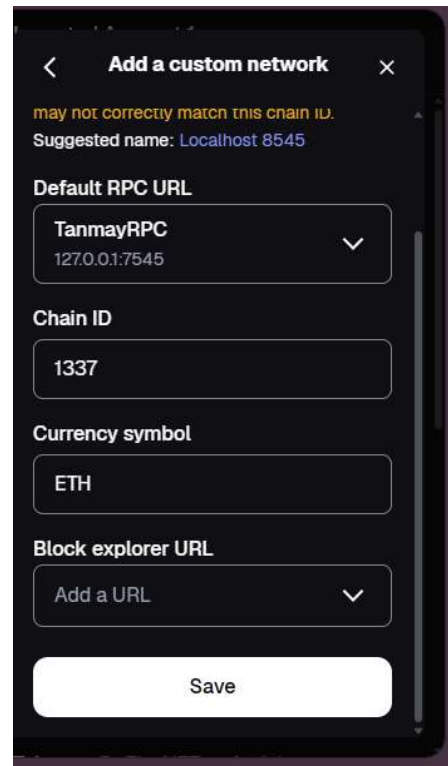
Transaction Hash

Ganache Address

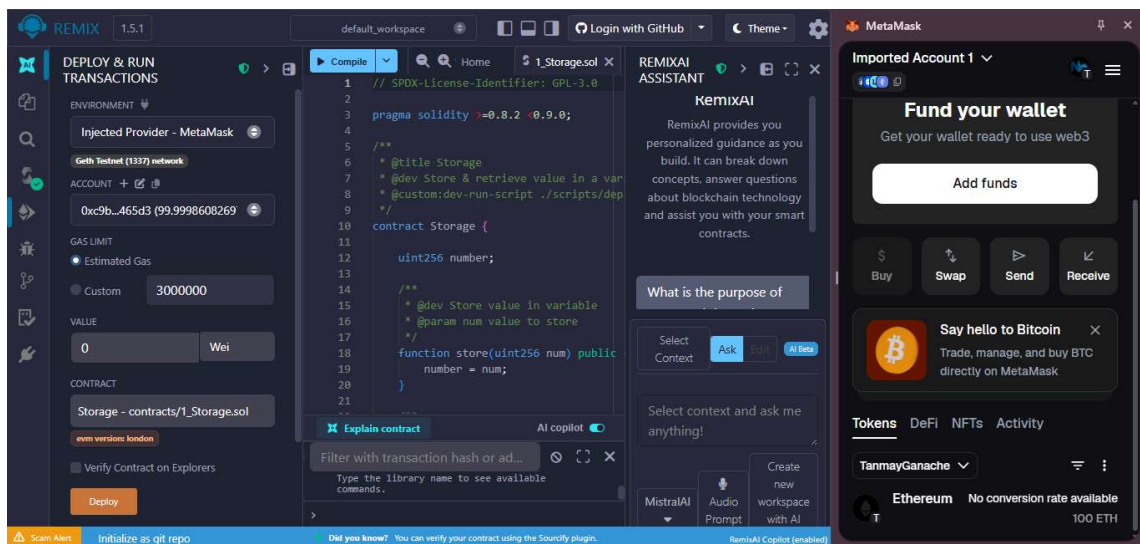
CURRENT BLOCK 1	GAS PRICE 2000000000	GAS LIMIT 6721975	HARDFORK MERGE	NETWORK ID 5777	RPC SERVER HTTP://127.0.0.1:7545	MINING STATUS AUTOMINING	WORKSPACE QUICKSTART	SAVE	SWITCH	⚙️
MNEMONIC							HD PATH m44'60'0'0account_index			
match garden exit tower misery volcano novel often unit bean vocal cricket										
ADDRESS 0xC9b16731f10547D024e7341644c20C97CDB465d3				BALANCE 100.00 ETH		TX COUNT 1		INDEX 0		



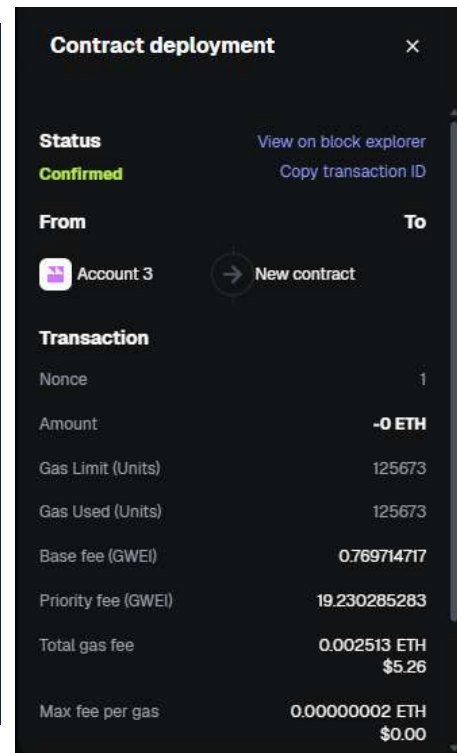
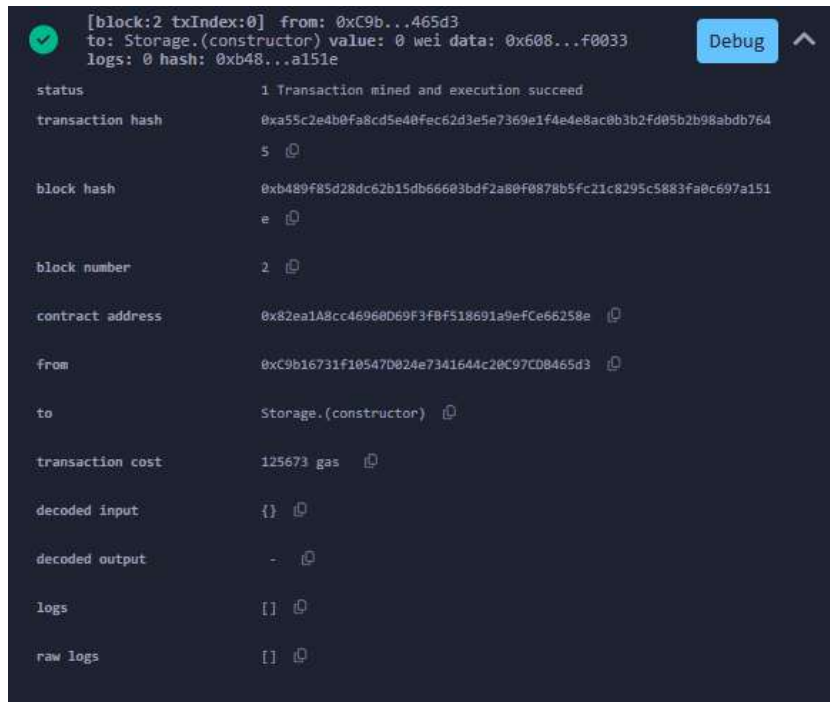
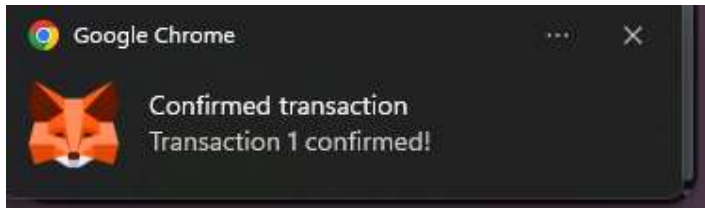
Importing ganache account to metamask



Configuring the RPC from ganache to metamask



Connected



Smart contract deployed

POST EXPERIMENT:

```
// SPDX-License-Identifier: MIT
pragma solidity ^0.8.0;
```

```
contract SimpleBank {
    // 1. Mapping to store balances
    mapping(address => uint256) public balances;
    address public owner;

    constructor() {
        owner = msg.sender;
    }

    // 2. Deposit function - must be 'payable'
    function deposit() public payable {
        require(msg.value > 0, "Deposit amount must be greater than 0");
        balances[msg.sender] += msg.value;
    }

    // 3. Withdraw function - handles the 'address payable' issue
```

```

function withdraw(uint256 _amount) public {
    require(balances[msg.sender] >= _amount, "Insufficient balance");

    // Update balance before sending to prevent re-entrancy
    balances[msg.sender] -= _amount;

    // Use payable(msg.sender) to ensure the address can receive ETH
    (bool success, ) = payable(msg.sender).call{value: _amount}("");
    require(success, "Transfer failed.");
}

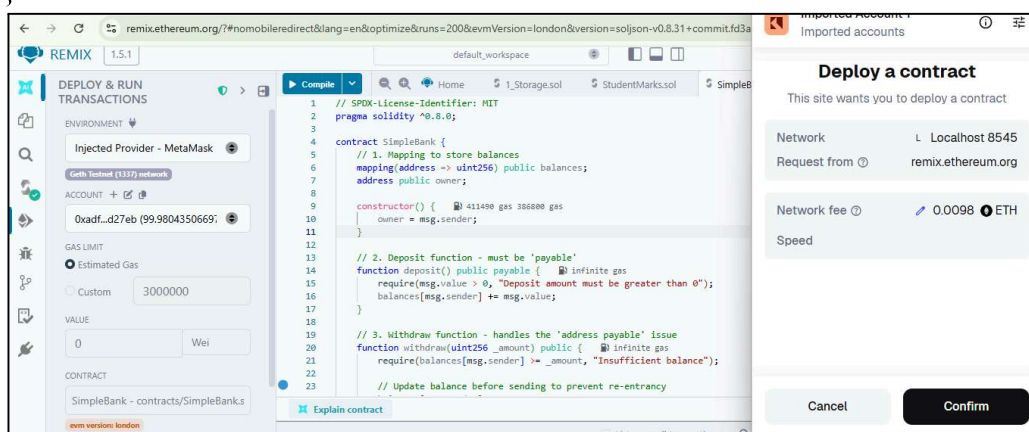
```

// 4. View function to check caller's balance

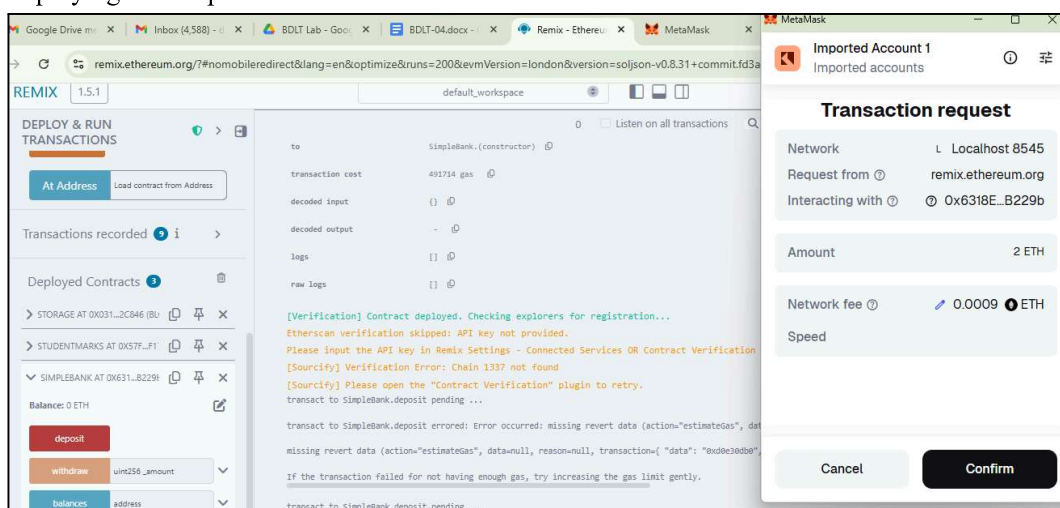
```

function getBalance() public view returns (uint256) {
    return balances[msg.sender];
}

```



Deploying the simple bank code



Depositing 2 ether as value and confirming the transaction

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the 'SimpleBank' contract at address 0x631...B229f. The balance is 2.0 ETH. The 'deposit' button is highlighted. On the right, the 'debug console' shows a successful transaction: '[call] from: 0xadfcb434eedfff52148abe900f1b0e85d35d27eb to: SimpleBank.getBalance() data: 0x120...65fe0'. The transaction hash is 0xace7b2583d3a372d3a9422d615ec52a38d69919422c5b549aef974d255b88cb. The block hash is 0xc57efb23e96f26aaff6a185a71d299a49b11b593405cfdb2ab7d4331c30fe9. The block number is 8. The transaction cost is 43675 gas. The decoded input is an empty object, and the decoded output is an empty object. The logs show a single log entry: '1 Transaction mined and execution succeed'. The raw logs show a single log entry: '1 Transaction mined and execution succeed'.

Get balance as 2 ether or 2,000,000,000,000,000,000 wei

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the 'SimpleBank' contract at address 0x631...B229f. The balance is 2.0 ETH. The 'owner' button is highlighted. On the right, the 'debug console' shows a successful transaction: '[call] from: 0xadfcb434eedfff52148abe900f1b0e85d35d27eb to: SimpleBank.owner() data: 0x8da...5cb5b'. The transaction hash is 0xace7b2583d3a372d3a9422d615ec52a38d69919422c5b549aef974d255b88cb. The block hash is 0xc57efb23e96f26aaff6a185a71d299a49b11b593405cfdb2ab7d4331c30fe9. The block number is 8. The transaction cost is 43675 gas. The decoded input is an empty object, and the decoded output is an empty object. The logs show a single log entry: '1 Transaction mined and execution succeed'. The raw logs show a single log entry: '1 Transaction mined and execution succeed'.

Check owner and get balance and address

The screenshot shows the Remix IDE interface. On the left, the 'DEPLOY & RUN TRANSACTIONS' panel displays the 'SimpleBank' contract at address 0x631...B229f. The balance is 2.0 ETH. The 'withdraw' button is highlighted. On the right, the 'debug console' shows a successful transaction: '[call] from: 0xadfcb434eedfff52148abe900f1b0e85d35d27eb to: SimpleBank.withdraw() data: 0x8da...5cb5b'. The transaction hash is 0xace7b2583d3a372d3a9422d615ec52a38d69919422c5b549aef974d255b88cb. The block hash is 0xc57efb23e96f26aaff6a185a71d299a49b11b593405cfdb2ab7d4331c30fe9. The block number is 8. The transaction cost is 43675 gas. The decoded input is an empty object, and the decoded output is an empty object. The logs show a single log entry: '1 Transaction mined and execution succeed'. The raw logs show a single log entry: '1 Transaction mined and execution succeed'.

Withdraw 1 ether and confirm the transaction

Genache

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
9

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

BLOCK
8

MINED ON
2026-02-09 10:24:32

GAS USED
43675

1 TRANSACTION

BLOCK
7

MINED ON
2026-02-09 10:18:51

GAS USED
491714

1 TRANSACTION

BLOCK
6

MINED ON
2026-02-09 10:04:29

GAS USED
136638

1 TRANSACTION

BLOCK
5

MINED ON
2026-02-09 09:59:03

GAS USED
641405

1 TRANSACTION

BLOCK
4

MINED ON
2026-02-09 09:49:03

GAS USED
23836

1 TRANSACTION

BLOCK
3

MINED ON
2026-02-09 09:48:38

GAS USED
43736

1 TRANSACTION

BLOCK
2

MINED ON
2026-02-09 09:46:50

GAS USED
125673

1 TRANSACTION

BLOCK
1

MINED ON
2026-02-09 09:33:22

GAS USED
125673

1 TRANSACTION

BLOCK
0

MINED ON
2026-02-09 09:20:17

GAS USED
0

NO TRANSACTIONS

On ganache application view the blocks create

Goerli

ACCOUNTS

BLOCKS

TRANSACTIONS

CONTRACTS

EVENTS

LOGS

SEARCH FOR BLOCK NUMBERS OR TX HASHES

CURRENT BLOCK
9

GAS PRICE
20000000000

GAS LIMIT
6721975

HARDFORK
MERGE

NETWORK ID
5777

RPC SERVER
HTTP://127.0.0.1:7545

MINING STATUS
AUTOMINING

WORKSPACE
QUICKSTART

SAVE

SWITCH

TX HASH

0×ace7b2503dfa372d3a9422de15ec52a389d69919422c5b549aef074d255b80cb

CONTRACT CALL

FROM ADDRESS
0×ADFCb434EdFFf52148aBE900f1B0e85D35d27E8

TO CONTRACT ADDRESS
0×6318E6A2A85248630702eDdD1AD3f2B8E038229b

GAS USED
43675

VALUE
200000000000000000

TX HASH

0×e9fa4ab10a33e8faddbb8ea3203b9df02dcbd9d75ddb7c6742de007ed6ecfed6

CONTRACT CREATION

FROM ADDRESS
0×ADFCb434EdFFf52148aBE900f1B0e85D35d27E8

CREATED CONTRACT ADDRESS
0×6318E6A2A85248630702eDdD1AD3f2B8E038229b

GAS USED
491714

VALUE
0

TX HASH

0×ed7bb40991960a552161f0bf9bc75043a2378eacd0f2a067d8f551ff65be164a

CONTRACT CALL

FROM ADDRESS
0×ADFCb434EdFFf52148aBE900f1B0e85D35d27E8

TO CONTRACT ADDRESS
0×57f430Ef80DdCb777365972cA0eb8080D30F17F9

GAS USED
136638

VALUE
0

TX HASH

0×ed7bb40991960a552161f0bf9bc75043a2378eacd0f2a067d8f551ff65be164a

CONTRACT CALL

FROM ADDRESS
0×ADFCb434EdFFf52148aBE900f1B0e85D35d27E8

TO CONTRACT ADDRESS
0×57f430Ef80DdCb777365972cA0eb8080D30F17F9

GAS USED
136638

VALUE
0